

Preparación de datos Couturier para construir la matriz de firmas

Cargar paquetes necesarios

```
library(tidyverse)
library(SummarizedExperiment)
library(rstatix)
library(ggpubr)
library(dplyr)
library(Seurat)
library(Matrix)
library(tibble)
```

Cargar couturier

```
couturier <- readRDS ("Couturier2020.rds")
#Objeto clase seurat
#54461 muestras
```

1. Crear nivel mixto

```
ifelse(couturier$Level_2 == "Tumor", as.character(couturier$Level_3),
       couturier$Level_2) %>% table()
nivel_mixto <- ifelse(couturier$Level_2 == "Tumor",
                      as.character(couturier$Level_3), couturier$Level_2)
couturier$Level_Mixto <- nivel_mixto
couturier@meta.data
glimpse(couturier@meta.data)

#Comprobación
unique(couturier@meta.data$Level_2)
unique(couturier@meta.data$Level_Mixto)
couturier@meta.data[, c("Level_2", "Level_Mixto")] %>% table()
couturier@meta.data[, c("Level_Mixto")] %>% table()
couturier@meta.data[, c("Level_2", "Level_Mixto")]
couturier@meta.data[couturier@meta.data$Level_2 ==
                     "Tumor", c("Level_2", "Level_Mixto")]

class(nivel_mixto)
length(nivel_mixto) #54461 células
```

2. Crear subset

```
# Extraer metadata como dataframe
meta <- couturier@meta.data
```

```

# Seleccionar 200 células al azar por categoría en Level_mixto
subset_ids <- split(rownames(meta), meta$Level_Mixto) %>%
  lapply(function(x) sample(x, size = min(200, length(x)))) %>%
  unlist() #Vuelve el resultado un vector

# Subset del objeto Seurat usando los IDs válidos
couturier_subset <- subset(couturier, cells = subset_ids)
#Se crea un nuevo objeto con las células seleccionadas anteriormente

dim(couturier) #14658 x 54461 células
dim(couturier_subset) #14658 x 2000 células
length(couturier_subset$Level_Mixto) #2000 células

```

3. Filtrar los genes

```

ivy_counts_df <- read.table("Ivy_CPM.txt", header = TRUE, sep = "\t")

sum(ivy_counts_df$Gene %in% rownames(couturier_subset))
#hay 12895 genes de ivygap en couturier
genes_comunes <- ivy_counts_df$Gene %in% rownames(couturier_subset)
#genes_comunes será un vector lógico que indica si los genes de ivy_counts_df
#están en couturier_subset

filtered_cout <- couturier_subset[rownames(couturier_subset)
                                %in% ivy_counts_df$Gene, ]
#filtra couturier_subset para quedarse solo con los genes presentes en
#ivy_counts_df$Gene

length(ivy_counts_df$Gene) #16694
nrow(filtered_cout@assays$RNA) #12895 Los mismos que coinciden con ivygap
nrow(couturier@assays$RNA) #14658

#Hemos reducido alrededor de 2000 genes
#Filtré couturier con IvyGap
#Los genes de couturier son los que hay en ivygap
#Pero ivygap aun tiene genes que no estan en couturier

```

4. Obtener los counts

```

#Acceder a los counts
counts <- GetAssayData(filtered_cout, assay = "RNA", layer = "counts")
class(counts) #dgCMatrx -> es una matriz dispersa

#Convertirlos los counts en matriz
cout_counts = as.matrix(GetAssayData(filtered_cout, assay = "RNA",
                                     layer = "counts"))

rownames(cout_counts) #Nombres de los genes
colnames(cout_counts) #Nombres de las células
nrow(cout_counts) #12895 genes
ncol(cout_counts) #2000 células, la misma longitud que en el metadata

#Convertir matriz en df

```

```
# Convertimos la matriz a dataframe
cout_counts_df <- as.data.frame(cout_counts)
cout_counts_df[1:2, 1:2]
dim(cout_counts_df) #12895 x 2000
```

5. Combinar metadata (fenotipos) con counts usando cbind()

```
# Agregamos los fenotipos como nombres de columna (excepto la primera columna)
ncol(cout_counts_df) #2000 celulas
length(couturier_subset$Level_Mixto) #2000 fenotipos

nivel_mixto <- couturier_subset$Level_Mixto
length(nivel_mixto) #2000

colnames(cout_counts_df) <- nivel_mixto
cout_counts_df[1:5, 1:5]
rownames(cout_counts_df) #genes
colnames(cout_counts_df) #fenotipos celulares
```

6. Añadir “GeneSymbol”

```
# Guardar los nombres originales
original_colnames <- colnames(cout_counts_df)
original_colnames

# Hacer los nombres únicos temporalmente
colnames(cout_counts_df) <- make.unique(original_colnames)

# Agregar la columna "GeneSymbol"
cout_counts_df <- cout_counts_df %>% rownames_to_column("GeneSymbol")
cout_counts_df[1:5, 1:5]

# Restaurar los nombres originales (después de la columna "GeneSymbol")
colnames(cout_counts_df)[-1] <- original_colnames

# Verificar el dataframe
head(cout_counts_df)
cout_counts_df[1:5, 1:5]
any(duplicated(colnames(cout_counts_df))) #TRUE
duplicated_columns <- colnames(cout_counts_df)[duplicated(colnames(cout_counts_df))]
duplicated_columns
str(cout_counts_df)
```

7. Requisitos de cibersort

```
#Tab-delimited tabular input format (.txt) with no double quotations
#and no missing entries.
any(is.na(cout_counts_df)) #FALSE
any(cout_counts_df == "") #FALSE

#Gene symbols in column 1; Reference cell phenotype labels in row 1.
```

```

cout_counts_df[1:5, 1:5]

#Cells with the same phenotype should have the same phenotypic label.
any(duplicated(colnames(cout_counts_df))) #TRUE
duplicated_columns <- colnames(cout_counts_df)[duplicated(colnames(cout_counts_df))]
duplicated_columns
print(colnames(cout_counts_df))

#No redundant genes
any(duplicated(rownames(cout_counts_df))) #FALSE

#Remove any non-assigned cells before uploading the file to CIBERSORTx.

#CIBERSORTx will automatically normalize the input data such that the sum of all
#normalized reads are the same for each transcriptome. If a gene
#length-normalized expression matrix is provided (e.g., RPKM), then the signature
#matrix will be in TPM (transcripts per million).
#If a count matrix is provided, the signature matrix will be in CPM (counts per
#million).
#Regardless of the input, the signature matrix and mixture files should be
#represented in the same normalization space.

#Data should be in non-log space. Note: if maximum expression value is <50;
#CIBERSORTx will assume that data are in log space, and will anti-log all
#expression values by 2x.

sapply(cout_counts_df, class) #Gene es character
max_value <- max(sapply(cout_counts_df,
  function(col) if (is.numeric(col)) max(col, na.rm = TRUE) else NA),
  na.rm = TRUE)
print(max_value) #3366

#CIBERSORTx performs a feature selection and therefore typically does not use
#all genes in the signature matrix. It is generally ok if some genes are missing
#from the user's mixture file. If less than 50% of signature matrix
#genes overlap, CIBERSORTx will issue a warning.
ivy_counts_df[1:5,1:5]
cout_counts_df$Gene
ivy_counts_df$Gene
mean(cout_counts_df$GeneSymbol %in% ivy_counts_df$Gene) #100%
mean(ivy_counts_df$Gene %in% cout_counts_df$GeneSymbol) #77.2%

```

8. Guardar el dataframe final como un archivo .txt

```
write_tsv(cout_counts_df, "Cout_counts_remix1.txt")
```